VRCCS-AC: Reinforcement Learning for Service Migration in Vehicular Edge Computing Systems

Zhen Gao, Lei Yang, and Yu Dai

***Abstract*—Existing service migration approaches provide minimal service delay for mobile vehicles (MVs) in vehicular edge computing (VEC) systems. Nonetheless, existing approaches focus more on formulating migration strategies rely on global information of the system, which may incur high signaling overhead and poor scalability. Furthermore, existing approaches are difficult to reuse previous migration strategies and necessitate significant interaction to adapt to new VEC scenarios. In this paper, we present a decentralized service migration approache base on Variational Recurrent and Critic-Coached Strategy reuse Actor-Critic (VRCCS-AC). First, a variational recurrent model (VRM) is introduced to efficiently obtain information from MV's local state through modeling VEC scenarios. An actor-critic enhances migration strategies by accessing both VEC scenario and VRM. Second, CCS leverages the critic-network to assess and select source service migration strategy. In every state, CCS selects the source strategy that exhibits the most significant one-step enhancement compared to the current target strategy, and develops a coached strategy. Then, the target strategy is regularized to imitate the coached strategy to facilitate effective strategy search and efficient strategy transfer. Experiments on the real-world datasets demonstrate that compared to the baselines, VRCCS-AC reduces latency by 10.11%∼18.57% and can quickly transfer to new VEC scenarios.**

***Index Terms*—Vehicular edge computing (VEC), service migration, reinforcement learning (RL), variational recurrent model (VRM), transfer learning.**

I. Introduction

As THE Internet of Things (IoT) expands, smart in-vehicle applications, such as automated navigation, multimedia entertainment, and radar positioning systems, become increasingly popular in smart vehicles. These applications demand substantial computational resources for real-time execution, which frequently exceeds the constrained computing capabilities of mobile vehicles (MVs). Fortunately, Vehicular Edge Computing (VEC) as the application of Multi-access Edge Computing (MEC) in the field of in-vehicle networks can better solve this issue. VEC provides MVs with sufficient computing resources and dramatically reduces the execution latency of offloaded tasks from the in-vehicle application. Typically, an in-vehicle application empowered by the VEC comprises a front-end module operating on MVs, along with a back-end service responsible for executing the offloaded tasks on an edge server (ES). However, when taking into account the mobility of MVs and the restricted coverage of ESs, the communications between an MV and the MV service operating on an ES might traverse multiple intermediate steps, impacting the task completion delay significantly. To tackle this issue, the service can be dynamically migrated to a more appropriate ES in order to uphold the desired task completion delay. Unfortunately, developing an ideal service migration strategy is extremely difficult due to the intricate dynamics of the VEC system and the mobility of MVs.

Several previous studies have put forward service migration algorithms, such as [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13]. Most of these approaches focus more on formulating migration strategies that rely on the system's global information, such as the load and resource allocation of ESs, the profile of offloading tasks, VEC network conditions, and the MVs' mobility. Therefore, these studies proposed that centralized service migration algorithms be deployed at ESs or cloud computing centres to provide server migration decisions for all MVs. While these methods have demonstrated commendable performance in service migration, some challenges and limitations still need to be addressed, as follows. First, in the practical VEC system, collecting global information of the VEC system typically incurs high signalling overhead [10], [11]. Second, these centralized service migration algorithms will result in poor scalability given that their computational complexity increases swiftly alongside the growing number of MVs [1], [2], [3], [4], [5], [6], [7], [8], [9].

Recently, some decentralized service migration approaches have also been proposed. For example, in [14], [15], the multi-armed bandit (MAB)-based service migration solution was proposed. These approaches simplify the VEC system scenario dynamics by employing the MAB model, overlooking the intrinsically enormous state dimension and intricate state transition in actual VEC scenarios, which could result in poor performance. Moreover, in [16], the authors modelled the service migration problem as a partially observable Markov decision process (POMDP) and proposed a multi-agent deep Q-learning-based service migration and mobility optimization algorithm. This algorithm adopts a centralized training and distributed execution (CTDE) mechanism. In the centralized

Manuscript received 27 January 2024; revised 18 May 2024; accepted 26 May 2024. Date of publication 30 May 2024; date of current version 30 December 2024. This work was supported by the National Key Research and Development Plan of China under Grant 2021YFF0901205. *(Corresponding author: Lei Yang.)*

Zhen Gao and Lei Yang are with the School of Computer Science and Engineering, Northeastern University, Shenyang 110032, China (e-mail: gaozhen610351@163.com; yanglei@mail.neu.edu.cn).

Yu Dai is with the School of College of Software, Northeastern University, Shenyang 110032, China (e-mail: daiy@swc.neu.edu.cn).

Digital Object Identifier 10.1109/TSC.2024.3407581

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

training phase, reinforcement learning (RL) agents can solve partially observable problems by accessing the global information of the VEC system. However, during the distributed execution phase, MVs still need help to obtain the load or resource allocation of ESs, making it difficult to make optimal migration decisions. Furthermore, in these service migration studies [17], [18], the computing resource that MV receives from an ES is not contingent on the quantity of tasks offloaded to the ES. In practice, however, ESs may have limited processing power, so the computational resource that an ES assigns to an MV is contingent upon the ES's load level (i.e., the total tasks offloaded to the ES). An ES may undergo a significant workload as many MVs migrate their services to it; hence, those offloaded tasks encounter long handling delays. Usually, the load of ES is difficult to predict, so it is a big challenge for each MV to determine its migration strategy independently.

To address the partially observable problem in the VEC system, we propose a new variational recurrent model-based service migration approach, which models the local state and rewards of MVs by utilising a latent stochastic variable. Our approach is based on an improved version of the VRNN [19] by introducing the service migration strategy of MV. Our approach is implemented based on the actor-critic framework. Specifically, a variational recurrent model is first presented to attain improved data efficiency through modelling the VEC scenarios. Then, an actor-critic enhances migration strategies by accessing both the VEC scenario and VRM.

Moreover, although these service migration schemes mentioned above [1], [2], [3], [4], [5], [6], [7], [8], [9], [14], [15], [20], [21], [22], [23], [24] can achieve highly competitive migration performance when these trained service migration schemes come across a new VEC scenario, various factors such as the transmission rate, task arrival rate, computing power and coverage of ESs, and other environmental parameters may vary significantly. As a result, the migration strategies obtained by these algorithms cannot be reused. They must be optimized from scratch to adapt to new VEC scenarios, resulting in substantial computational and sampling expenses. Recently, a few studies have begun to investigate the problem of service migration strategy reuse (how to effectively reuse source service migration strategy to accelerate target service migration strategy learning), e.g., [25], [26], [27], [28]. These approaches tackle the migration strategy reuse problem by incorporating new network structures into the basic RL framework, e.g., hierarchical high-level service migration strategies over source service migration strategies [27] or evaluations of source service migration strategies' value-network for the target environment [25], [26], [28]. Nevertheless, training these additional network architectures dramatically reduces the efficiency of service migration strategies reuse. This is because the hierarchical architecture will destroy the stability of the optimization, and the computation of the value network of the source service migration strategy leads to high computational and sampling overheads.

To avoid introducing new network structures, we propose a new service migration strategy reuse method that employs the critic-network to select source service migration strategies. Our approach is implemented based on the actor-critic framework. Specifically, we introduce a Critic-network Coached service Migration Strategy reuse algorithm (CCS), which circumvents the need for training additional components and effectively reuses source service migration strategies. CCS leverages the critic-network, a common component found in actor-critic algorithms, to assess and select source service migration strategies. In every state, CCS selects the source service migration strategy that exhibits the most significant one-step enhancement compared to the current target service migration strategy, creating a coached service migration approach. Then, the target service migration strategy is regularized to mimic the coached service migration strategy to perform an efficient service migration strategy search and achieve efficient service migration strategy migration. The contrast of current approaches with the VRCCS-AC algorithm through the critical features is outlined in Table I. The following are the paper's primary contributions:

- *The Dynamic Service Migration Problem in Partially Observable VEC Scenario:* We formulate the service migration of the VEC scenario as a POMDP problem for non-divisible and delay-sensitive tasks. In the case of incomplete VEC system information and unknown computational loads at ESs, the problem strives to minimize the anticipated long-term cost associated with tasks (including the service migration latency, the computational latency of tasks, and the communication latency).
- *VRCCS-AC Algorithm-based Service Migration Solution:* We present a decentralized service migration method based on the VRCCS-AC algorithm. First, a VRM is proposed to efficiently obtain information from MV's local state through modelling VEC scenarios. An actor-critic enhances migration strategies by accessing both the VEC scenario and VRM. Second, CCS leverages the critic-network to assess and select source service migration strategy. In every state, CCS selects the source strategy that exhibits the most significant one-step enhancement compared to the current target strategy and develops a coached strategy. Then, the target strategy is regularized to imitate the coached strategy to facilitate effective strategy search and efficient strategy transfer.
- *Performance Evaluation:* Given the constrained computing power of MVs and ESs, we first consider that the Cloud Computing Center initially trains the proposed service migration solution. Then, each MV agent conducts distributed service migration strategies based on local states using the trained VRCCS-AC algorithm. Extensive simulation experiments demonstrate that VRCCS-AC dramatically decreases the latency compared to existing migration solutions. Moreover, VRCCS-AC exhibits transfer capabilities to new VEC scenarios.

Next, the structure of this paper is as follows. Section II reviews the related works. Section III provides the system model and problem formulation for service migration. In Section IV, we introduce the problem definition and problem formulation for service migration. Section V introduces the VRCCS-AC algorithm in detail. In Section VI, we assess the performance of the VRCCS-AC algorithm. Section VII concludes the paper.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF EXISTING METHODS WITH THE VRCCS-AC ALGORITHM

Key Features \ Algorithms	[1]–[13]	[14], [15]	[16]	[20]–[24]	[17], [18]	[25]–[28]	**VRCCS-AC Algorithm**
Global Observation VEC Environment	✓						
Partial Observation VEC Environment		✓	✓				✓
Multi-Armed Bandit Algorithm		✓					
Q-Learning Algorithm			✓				
VRM-based Actor-Critic Method							✓
Adequate Computing Resources at ESs					✓		
Unknown Load Level Dynamics at ESs							✓
Static VEC Environment	✓	✓	✓	✓	✓		
Dynamic VEC Environment						✓	✓
Introducing Extra Components						✓	
Critic-Coached Strategy Reuse (No need to introduce extra components)							✓

II. RELATED WORKS

Based on the placement of the service migration decision agent, service migration approaches can be categorized into centralized and decentralized schemes. Moreover, we further explore the existing service migration schemes based on transfer learning algorithms.

A. The Centralized Service Migration Approaches

Chen et al. [2] studied dynamic service migration and request routing for microservice in multicell MEC and proposed a Lyapunov optimization-based algorithm. Wang et al. [6] proposed a proactive service migration solution based on Lyapunov optimization and spatial-temporal mobility prediction in VEC scenarios. Shah et al. [4] introduced a service mobility management method using software-defined networking (SDN)-based in VEC. Zhou et al. [5] studied energy-efficient service migration for multi-user heterogeneous dense cellular networks and proposed a Lyapunov and particle swarm optimizations-based algorithm. By extending the MARL approach, Liu et al. [1] introduced a distributed task migration optimization solution in MEC. Ren et al. [7] proposed a DRL algorithm based on blockchain-based VEC network trust management for vehicular service offloading and migration. Wang et al. [8] proposed an RL approach for delay-aware microservice coordination in MEC. Chen et al. [3] proposed a dynamic task allocation and service migration solution based on RL in an edge-cloud IoT system. Chen et al. [9] proposed a mobility-aware service migration based on a data-driven approach for seamless provision. These approaches focus more on formulating migration strategies that rely on the system's global information. Nevertheless, gathering real-time comprehensive system-level information in a practical VEC scenario imposes significant communication overhead [10], [11]. Moreover, these centralized control methods are susceptible to scalability, given that their computational complexity increases swiftly alongside the growing number of MVs.

B. The Decentralized Service Migration Approaches

Sun et al. [14] introduced a service placement solution based on the Multi-Armed Bandit (MAB) algorithm in the VEC system scenario. Ouyang et al. [15] proposed an adaptive user-managed service placement solution based on an online learning approach for MEC. These approaches simplify the VEC system scenario dynamics by employing the MAB model, overlooking the intrinsically enormous state dimension and intricate state transition in actual VEC scenarios, which could result in poor performance. Moreover, Yuan et al. [16] modelled the service migration problem as Dec-POMDP and proposed a multi-agent deep Q-learning-based service migration and mobility optimization algorithm. This algorithm adopts a CTDE mechanism. In the centralized training phase, RL agents can solve partially observable problems by accessing the global information of the VEC system. However, during the distributed execution phase, MVs still need help to obtain the load or resource allocation of ESs, making it difficult to make optimal migration decisions. Furthermore, Cao et al. [17] proposed a cost-aware service migration path selection method based on DRL. Yu et al. [18] proposed a hybrid learning based service migration method for cost minimization with deadlines in multi-user MEC systems. In these studies, the computing resource that MV receives from an ES is not contingent on the quantity of tasks offloaded to the ES. In practice, however, ESs may have limited processing power, so the computational resource that an ES assigns to an MV is contingent upon the ES's load level (i.e., the total tasks offloaded to the ES). An ES may undergo a significant workload as many MVs migrate their services to it; hence, those offloaded tasks encounter long handling delays. Usually, the load of ES is difficult to predict, so it is a big challenge for each MV to determine its migration strategy independently.

C. Transfer Learning-Based Service Migration Approaches

Hummaida et al. [27] proposed a hierarchical decentralized architecture to enable adaptive, scalable virtual machine migration. Zhao et al. [26] studied the reinforced-lstm trajectory prediction-driven dynamic service migration solutions. Peng et al. [25] proposed a computing and communication cost-aware service migration solution enabled by transfer RL for the VEC system. Shuai et al. [28] proposed a transfer RL-based task scheduling solution in the MEC system. These approaches tackle the migration strategy reuse problem by incorporating new network structures into the basic RL framework, e.g., hierarchical high-level service migration strategies over source service migration strategies [27] or evaluations of source service migration

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

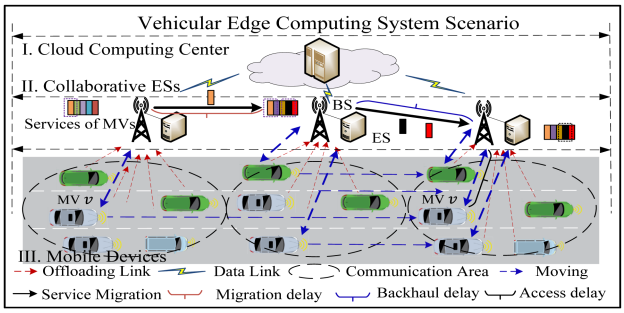

Fig. 1. Some examples of service migration in three-level framework-based VEC system.

strategies' value-network for the target environment [25], [26], [28]. Nevertheless, training these additional network architectures dramatically reduces the efficiency of service migration strategies reuse. This is because the hierarchical architecture will destroy the stability of the optimization, and the computation of the value network of the source service migration strategy leads to high computational and sampling overheads.

III. System Model

As illustrated in Fig. 1, the VEC system scenario comprises the Cloud Computing Center layer, Collaborative ESs layer, and Mobile Devices layer. Specifically, first, we consider that the Cloud Computing Center has sufficient computational and communication capabilities. Given the constrained computing power of MVs and ESs, we first consider that the Cloud Computing Center initially trains the proposed service migration solution. Then, each MV agent conducts distributed service migration strategies based on local states using the trained VRCCS-AC algorithm. Moreover, in this VEC system scenario, MVs can migrate services to any ES. Thus, at the Collaborative ESs layer, there is a collection of base stations (BSs) and ESs that are responsible for managing the offloading tasks from the MVs, facilitating the services of the MVs, and supporting communication services to the MVs. The ESs are represented by $\mathcal{N} = \{1, 2, \ldots, N\}$. Finally, the Mobile Devices layer encompasses a lot of heterogeneous MVs, $\mathcal{V} = \{1, 2, \ldots, V\}$, within a geographic region covered by a cluster of ESs, each equipped with various computational resources. In this VEC scenario, we consider a time-slotted model-based system where each time slot lasts for a duration of δ. We describe the fundamental symbols used in this paper in Table II.

We specify the ES that upholds the service of an MV v in time slot t as the serving ES for the MV, expressed as $ser_v(t)$, and the ES that straight connects with the MV in time slot t as the local ES for the MV, expressed as $loc_v(t)$. Typically, the ESs are interlinked by reliable backhaul connections, enabling the MV to maintain access to its service through multi-hop communication among ESs, even in the absence of a direct connection to the serving ES. Nevertheless, to guarantee acceptable QoS, the service of the MVs must be flexibly migrated among the ESs based on the MV changes its location (i.e., if $ser_v(t) \neq loc_v(t)$, it becomes necessary to formulate a service migration decision for the MV v during the time interval t. Next, we provide a detailed introduction to the migration communication model, computation model, and migration model in the proposed three-level framework-based VEC system scenario.

TABLE II
A Summary of Key Notations

Symbols	Descriptions
V	Number of MVs.
N	Number of ESs.
$ser_v(t)$	The serving ES of MV v.
$loc_v(t)$	The connected ES of MV v.
$\tau_v^{md}(t)$	Service migration latency of MV v.
$d_v^s(t)$	The data size of the MV's service.
$D^{sl}(t)$	The distance between ES $ser_v(t)$ and ES $loc_v(t)$.
$b_n(t)$	Migration route's bandwidth.
$\sigma_n(t)$	Coefficient of the migration latency.
$d_v^o(t)$	Offloaded task's data size.
$\lambda_v(t)$	Offloaded task's processing density.
$\kappa_v(t)$	CPU cycles required of the offloaded task.
$\tau_v^d(t)$	Deadline of the task.
$\chi_{a_v(t)}$	Load level of the serving ES.
$F_{a_v(t)}$	Computating power of the serving ES.
$\tau_v^{od}(t)$	Processing delay for the offloaded task.
$r_v^{tr}(t)$	Uplink transmission rate.
B	Bandwidth between MV and the local ES.
p_v	Transmission power of the MV.
$g_v(v, loc_v(t))$	Channel gain between MV and the local ES.
$\tau_v^{tr}(t)$	Upload latency for offloaded tasks.
$d_v^{ls}(t)$	Hop distance between the serving ES and local ES.
$\tau_v^{bd}(t)$	The backhaul delay.
$b_v^{bh}(t)$	Bandwidth of backhaul network.
$\tau_v^c(t)$	Communication delay.
$\tau_v^{tot}(t)$	Total delay of MV v.

A. Migration Model

When the MV moves to a different location, the MV develops a migration policy for the present service and then offloads the computational tasks to the serving ES for performing. Define the migration policy at time slot t as $a_v(t)$, $(a_v(t) \in \mathcal{N})$, therefore indicating that $a_v(t)$ can be any of the ESs available in this region. We define migration delay as being generated when MV's services are transferred from a previous serving ES to another ES. As mentioned in these works [1], [29], [30], we consider that the migration delay is a non-decreasing function and is defined by the following (1),

$$\tau_v^{md}(t) = \begin{cases} 0, & \text{if } ser_v(t) = loc_v(t), \\ \frac{d_v^s(t)}{b_n(t)} + \varrho^n D^{sl}(t), & \text{if } ser_v(t) \neq loc_v(t). \end{cases} \quad (1)$$

where $D^{sl}(t)$ represents the distance between the current serving ES $ser_v(t)$ and the local ES $loc_v(t)$ for the MV v. $\varrho^n(t)$ represents a nonnegative coefficient of the migration latency. $b_n(t)$ denotes the network transmission bandwidth throughout the migration route. $d_v^s(t)$ indicates the data size of the service-related files for MV v to be migrated.

B. Computation Model

When the service of the MV v is migrated to the appropriate serving ES, the MVs can then delegate their computing tasks to the serving ES for execution. We assume that the computing resources of ES are limited and are shared by multiple MVs to perform the MV's offloaded tasks. At time slot t, the offloaded

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

task's data size for MV v is denoted as $d_v^o(t)$ and the offloaded task's processing density is denoted as $\lambda_v(t)$. Further, the CPU cycles required to process the offloaded task is denoted as $\kappa_v(t) = d_v^o(t) \cdot \lambda_v(t)$. $\tau_v^d(t)$ represents the deadline of the task. In other words, if the task is not completed within $\tau_v^d(t)$, it will be promptly discarded. Let $J_v(t) = \{\kappa_v(t), d_v^o(t), \tau_v^d(t)\}$ denote this offloaded task. In addition, we define the load level of the serving ES as $\chi_{a_v(t)}$ and the computing capabilities of the serving ES as $F_{a_v(t)}$. Hence, we define the processing latency for the offloaded task using the following (2),

$$\tau_v^{od}(t) = \frac{\kappa_v(t)}{\left(\frac{\kappa_v(t)}{\chi_{a_v(t)} + \kappa_v(t)} F_{a_v(t)}\right)} = \frac{\chi_{a_v(t)} + \kappa_v(t)}{F_{a_v(t)}}. \quad (2)$$

C. Communication Model

After migrating the MV's service to the appropriate ES, there is a communication delay when MVs upload the offloaded tasks to the serving ES. In general, the communication delay can be divided into two components: the transmission latency for offloaded tasks between the MV and the local ES, as well as the backhaul latency between the local ES and the serving ES. Further, similar to these works [1], [3], we define the upload rate of offloaded tasks from the MV to the local ES using the following (3),

$$r_v^{tr}(t) = B\log_2\left(1 + \frac{p_v |g_v(v, loc_v(t))|^2}{B\mathcal{N}}\right), \quad (3)$$

where B represents the wireless uplink bandwidth between MV and the local ES. p_v denotes the transmission power of the MV. $\mathcal{N}$ denotes the power spectral density of the additive white Gaussian noise. $g_v(v, loc_v(t))$ represents the channel gain between MV and the local ES. Further, the latency of uploading an offloaded task to the local ES can be defined by the following (4),

$$\tau_v^{tr}(t) = \frac{d_v^o(t)}{r_v^{tr}(t)}. \quad (4)$$

We consider that when the serving ES $ser_v(t)$ of MV v is different from the local ES, there will be a backhaul delay. Similar to these works [29], [30], the latency of the backhaul is mainly determined by the distance of each hop across the most direct route and the amount of the offloaded task data. The hop distance between the serving ES and local ES is denoted as $d_v^{ls}(t) = |ser_v(t) - loc_v(t)|^2$. Therefore, the backhaul delay can be denoted using the following (5),

$$\tau_v^{bd}(t) = \begin{cases} 0, & \text{if } d_v^{ls}(t) = 0, \\ \frac{d_u^o(t)}{b_v^{bh}(t)} + \varrho^{bh}(t) d_v^{ls}(t), & \text{if } d_v^{ls}(t) \neq 0. \end{cases} \quad (5)$$

where $\varrho^{bh}(t)$ represents a nonnegative scalar. $b_v^{bh}(t)$ denotes the network bandwidth of backhaul network. In particular, when the serving ES and the MV have a direct connection ($d_v^{ls}(t) = 0$), there is no backhaul expense. Therefore, the latency incurred in the communication model can be denoted using the following (6),

$$\tau_v^c(t) = \tau_v^{tr}(t) + \tau_v^{bd}(t). \quad (6)$$

IV. Problem Definition and Formulation

A. Problem Definition

We formulate the service migration of the VEC scenario as a POMDP problem for non-divisible and delay-sensitive tasks. In the case of incomplete VEC system information (e.g., the resource allocation or load of ESs, the mobility of MVs), the problem strives to minimize the anticipated long-term cost associated with tasks (i.e., total delay $\tau_v^{tot}(t) = \tau_v^{md}(t) + \tau_v^{od}(t) + \tau_v^{tr}(t)$). This problem can be mathematically expressed using the following (7),

$$\min_{\{a_v(1),\ldots,a_v(T)\}} \quad \sum_{t=1}^{T} \tau_v^{tot}(t) \quad (7)$$

$$\text{s.t.} \quad C_1 : \forall v \in \{1, \ldots, V\}, \quad (7a)$$

$$C_2 : a_v(t) \in \{1, \ldots, N\}, \quad (7b)$$

$$C_3 : \tau_v^{tot}(t) \leq \tau_v^d(t), \quad (7c)$$

where (7a) denotes that it is satisfied for all MVs. (7b) denotes the service migration strategy for MV. (7c) indicates the time limit for the MV's offloaded task $J_v(t)$.

B. MDP-Based Problem Formulation

In our proposed VEC system scenario, the variables that affect the migration strategy of current MVs mainly include the mobility of MVs, offloaded task information, load level of ESs, resource allocation of ESs, and other MVs. Typically, MVs can make optimal migration strategies when they have access to all relevant information within the VEC system. However, it is challenging for MVs to collect this information completely and in real time. For example, when an MV makes a migration strategy, the MV does not have prior knowledge of the load levels at the ESs, as the load is also influenced by the migration strategies of other MVs. Moreover, the load levels at the ESs may vary over time. In order to make an optimal service migration strategy based on partial state information in the VEC system, we consider modelling the problem as a POMDP, which empowers the MV agent to efficiently approximate the results of its actions when the state of the environment cannot be captured ultimately. We define the POMDP as a 5-tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma)$. $\mathcal{S}$ denotes a set of VEC system states. $\mathcal{A}$ denotes a set of actions (i.e., the service migration strategy of the MV agent). $\mathcal{P}$ denotes the state-transition probability function. R denotes the reward function. $\gamma \in [0, 1]$ indicates discount factor. we will further introduce the definition of the RL-based service migration problem as follows.

1) The Observation/Local State $\boldsymbol{o}_v(t)$ of MV v: According to our proposed system model in VEC, we consider MV as an agent and define its local state as the previous serving ES $ser_v(t-1)$, task information $J_v(t)$, and the upload rate of offloaded tasks from the MV to the local ES $r_v^{tr}(t)$. The local state by MV v can be expressed using the following (8),

$$\boldsymbol{o}_v(t) = \left(ser_v(t-1), J_v(t), r_v^{tr}(t)\right). \quad (8)$$

Further, let $\mathcal{S}(t) = \{\boldsymbol{o}_1(t), \ldots, \boldsymbol{o}_V(t)\}$ denote the joint local states of all MVs in time slot t.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

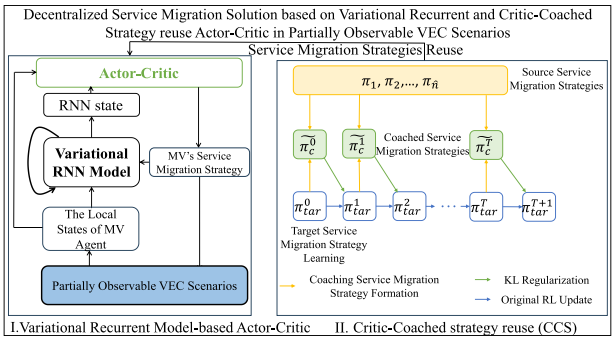

Fig. 2. The VRCCS-AC algorithm-based service migration solution.

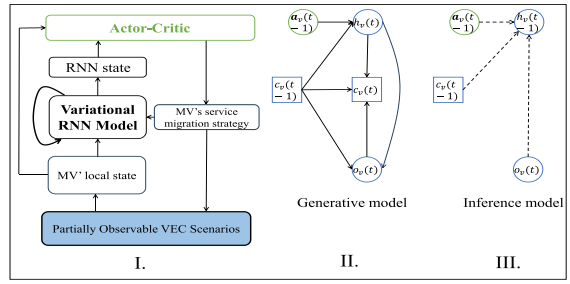

Fig. 3. The service migration solution based on the variational recurrent model-based actor-critic in partially observable VEC scenarios.

2) The Service Migration Strategy $\boldsymbol{a}_v(t)$ of MV v: Based on our VEC scenario, the service migration strategy (i.e., the action) of the MV agent is defined as $\boldsymbol{a}_v(t)$. $\boldsymbol{a}_v(t) \in \mathcal{A}$ denotes the ES where MV's service is migrated ($\boldsymbol{a}_v(t) \in \mathcal{N}$).

3) State-Transition Probability $\mathcal{P}$: We define $\mathcal{P}$ to denote the state-transition probability in the VEC scenario. $\mathcal{P}$ denotes the probability that the current state $\boldsymbol{o}_v(t)$ is transferred to the next state $\boldsymbol{o}_v(t+1)$ when the MV agent applies a service migration strategy $\boldsymbol{a}_v(t)$ to the VEC scenario.

4) The Reward $R_v(t)$ of MV v: Given the local state $\boldsymbol{o}_v(t)$, MV v utilizes the service migration strategy $\boldsymbol{a}_v(t)$ to engage with the VEC system and obtain the corresponding reward. Building upon the proposed VEC system, we define MV's reward as a value related to service migration, computation of offloaded tasks, and transmission of offloaded tasks delay. Further, MV's reward $R_v(t)$ is expressed through the following (9),

$$R_v(t) = \sum_{t=1}^{T} \gamma^t \cdot r(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t)). \tag{9}$$

We define $r(\boldsymbol{o}_v(t),\ \boldsymbol{a}_v(t)) = -(\eta_1 \tau_v^{md}(t) + \eta_2 \tau_v^{od}(t) + \eta_3 \tau_v^{tr}(t))$, where $\eta_1 \tau_v^{md}(t) + \eta_2 \tau_v^{od}(t) + \eta_3 \tau_v^{tr}(t)$ represents the costs related to service migration. η_1, η_2, and η_3 are the constants. As mentioned in these works [10], [31], [32], the offloading task is usually computationally intensive. However, the amount of task data is small. Therefore, for $\tau_v^{od}(t)$ and $\tau_v^{tr}(t)$, we will consider a larger weight for $\tau_v^{od}(t)$ (t). Both $\tau_v^{tr}(t)$ and $\tau_v^{md}(t)$ are costs in terms of transmission, and we consider the proportion of weights of similar sizes. The aim of our model is to optimize the reward $R_v(t)$ of each MV v agent over an extended period of time.

V. Proposed Solution: VRCCS-AC Based Service Migration Algorithm

As illustrated in Fig. 2, we present a decentralized service migration approach based on the VRCCS-AC algorithm. Existing service migration approaches supply exceptionally minimal service delay for MVs in VEC systems. Nonetheless, existing approaches focus more on formulating migration strategies that rely on the system's global information, which may incur high signalling overhead and poor scalability. To address this problem, a VRM is first proposed to attain improved data efficiency through modelling the VEC scenario. Additionally, an actor-critic controller is introduced to enhance strategies by accessing both VEC scenarios and VRM. Furthermore, existing approaches make it difficult to reuse previous migration strategies and necessitate significant interaction to adapt to new VEC scenarios. To address this problem, CCS first leverages the critic-network to assess and select a source service migration strategy. In every state, CCS selects the source strategy that exhibits the most significant one-step enhancement compared to the current target strategy and develops a coached strategy. Then, the target strategy is regularized to imitate the coached strategy to facilitate effective strategy search and efficient strategy transfer. Next, we will provide a detailed introduction to two optimization modules.

A. Variational Recurrent Model-Based Actor-Critic

As illustrated in Fig. 3, we propose the service migration solution based on the variational recurrent model-based actor-critic in partially observable VEC scenarios. To address the POMDP problem, a VRM is first proposed to efficiently obtain information from MV's local state through modelling VEC scenarios. Then, an actor-critic enhances migration strategies by accessing both the VEC scenario and VRM. Next, we will describe variational recurrent neural networks, variational recurrent state-transition models and actor-critic in detail.

1) Variational Recurrent Neural Networks: In order to construct state transitions that may have stochasticity and complexity, our approach is based on an improved version of the VRNN [19] by introducing the service migration strategy of MV. As illustrated in Fig. 3, our approach consists of a variational generative model and a variational inference model. It's a recurrent latent variable model capable of learning to encode and make predictions for complex sequential local states $\boldsymbol{o}_v(t)$ of MV agent using a random latent variable $h_v(t)$. The generative model forecasts future local states of MV agents based on their internal states,

$$\begin{aligned} &h_v(t) \sim \mathcal{B}(\mu_g(t), \\ &\mathrm{diag}(\sigma_g^2(t))), [\mu_g(t), \sigma_g^2(t)] = f^{\text{prior}}(c_v(t-1)), \\ &[\mu_{g'}(t), \sigma_{g'}^2(t)] = f^{\text{decoder}}(h_v(t), c_v(t-1)), \\ &\boldsymbol{o}_v(t)|h_v(t) \sim \mathcal{B}(\mu_{g'}(t), \mathrm{diag}(\sigma_{g'}^2(t))), \end{aligned} \tag{10}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

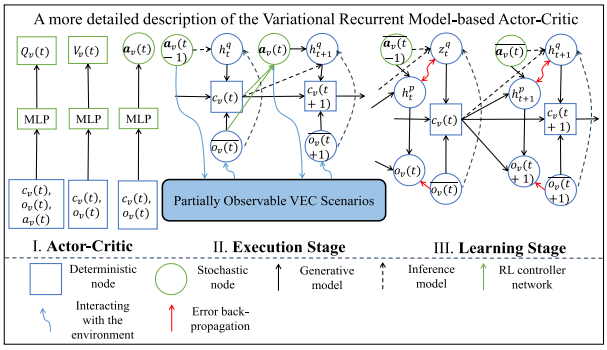

Fig. 4. A more detailed description of the variational recurrent model-based actor-critic. $\boldsymbol{a}_v(t)$: action; $h_v^q(t)$: latent variable; $c_v(t)$: RNN state variable; $o_v(t)$: local state of the MV agent (including reward); $Q_v(t)$: state-action value function; $V_v(t)$: state value function. A bar on a variable denotes the actual value from the experience cache pool or the VEC scenario. Each random variable is characterized by a parameterized Gaussian distribution with a diagonal covariance matrix.

where $f^{(\cdot)}$ is implemented by feed-forward neural networks. $c_v(t)$ denotes the state variable of the RNN. Further, the update of $c_v(t)$ is denoted using the following (11),

$$c_v(t) = f^{\text{RNN}}(c_v(t-1); h_v(t), \boldsymbol{o}_v(t)). \tag{11}$$

Based on $\boldsymbol{o}_v(t)$ and $c_v(t)$, the inference model represents the latent variable $h_v(t)$. Further, the inference model is defined using the following (12),

$$\begin{aligned} &h_v(t)|\boldsymbol{o}_v(t) \sim \mathcal{B}(\mu_h(t), \text{diag}(\sigma_h^2(t))), \\ &[\mu_h(t), \sigma_h^2(t)] = f^{\text{encoder}}(\boldsymbol{o}_v(t), c_v(t-1)). \end{aligned} \tag{12}$$

For time-series data encompassing T time intervals, the training process is carried out by maximizing the evidence lower bound $ELBO$. Similar to [33], we define $ELBO$ by the following (13),

$$\begin{aligned} ELBO = &\sum_t^T [-D_{KL}(g'(h_v(t)|h_v(t:t-1), \boldsymbol{o}_v(1:t))|| \\ &g(\boldsymbol{o}_v(t)|\boldsymbol{o}_v(1:t-1)))] + \mathbb{E}_{g'(h_v(t)|\boldsymbol{o}_v(1:t), h_v(t-1))} \\ &[\log(g(\boldsymbol{o}_v(t)|h_v(1:t), \boldsymbol{o}_v(1:t-1)))], \end{aligned} \tag{13}$$

where g and g' are probability density functions (PDFs) of $h_v(t)$ parameterized by the generative model and the inference model, respectively. In a POMDP, a VRNN can be utilized to model the VEC scenario and encode the underlying states using its state variable $h_v(t)$. Hence, a MV agent can gain advantages from a well-trained VRNN model since $c_v(t)$ offers supplementary information about the VEC scenario beyond the current local states $\boldsymbol{o}_v(t)$ of the MV agent.

2) Variational Recurrent State-Transition Models: A more detailed description of Fig. 3 is shown in Fig. 4. Our approach is based on an improved version of the VRNN [19] by introducing the service migration strategy of MV, meaning that service migration strategies executed by the MV agent are utilized in both the inference model and the generative model. Additionally, because we are representing state-transition and reward functions, we incorporate the reward $r_v(t-1)$ into the present local state $\boldsymbol{o}_v(t)$ of the MV agent. Therefore, we define the inference model I by the following (14),

$$\begin{aligned} &h_{v,I}(t)|\boldsymbol{o}_v(t) \sim \mathcal{B}(\boldsymbol{\mu}_I(t), \text{diag}(\boldsymbol{\sigma}_I^2(t))), \\ &[\boldsymbol{\mu}_I(t), \boldsymbol{\sigma}_I^2(t)] = I(\boldsymbol{o}_v(t), c_v(t-1), \boldsymbol{a}_v(t-1)), \end{aligned} \tag{14}$$

Further, we define the generative model based on the parameter I' by the following (15),

$$\begin{aligned} &h_v(t) \sim \mathcal{B}(\mu_{I'}(t), \text{diag}(\sigma_{I'}^2(t))), \\ &[\mu_{I'}(t), \sigma_{I'}^2(t)] = f^{\text{prior}}(c_v(t-1), \boldsymbol{a}_v(t-1)), \\ &[\mu_{g'}(t), \sigma_{g'}^2(t)] = f^{\text{decoder}}(h_v(t), c_v(t-1)), \\ &\boldsymbol{o}_v(t)|h_v(t) \sim \mathcal{B}(\mu_{g'}(t), \text{diag}(\sigma_{g'}^2(t))), \end{aligned} \tag{15}$$

where $c_v(t) = \text{LSTM}(c_v(t-1); h_v(t), \boldsymbol{o}_v(t))$. LSTM denotes the long-short term memory network.

While training a VRNN, the VRM is trained by maximizing an ELBO as depicted in Fig. 3(III). Further, we express this by the following (15),

$$\begin{aligned} ELBO = &\sum_t^T [\mathbb{E}_{qI}[\log gI'(\boldsymbol{o}_v(t))|h_v(1:t), \boldsymbol{o}_v(1:t-1)]] \\ &- D_{KL}[qI(h_v(t)|h_v(1:t-1), \bar{\boldsymbol{o}}_v(1:t), \bar{\boldsymbol{a}}_v(1:t)) \\ &||gI'(h_v(t)|h_v(1:t-1), \bar{\boldsymbol{o}}_v(1:t), \bar{\boldsymbol{a}}_v(1:t))]. \end{aligned} \tag{16}$$

3) Actor-Critic Controller: Our approach is implemented based on the actor-critic framework. As depicted in Fig. 3(I), we utilize multi-layer perceptrons (MLP) as function approximators for $V_v(t)$ and $Q_v(t)$, respectively. The inputs for the $Q(t)$ network comprise $(\boldsymbol{o}_v(t), c_v(t), \boldsymbol{a}_v(t))$, while $V(t)$ is derived from $(\boldsymbol{o}_v(t), c_v(t))$. Further, we use two Q networks λ_1 and λ_2 and compute $Q = \min(Q_{\lambda_1}, Q_{\lambda_2})$ in (5) and (7) for better performance and stability. Furthermore, we also used a target value network for computing V in (6). The policy function π_η follows a parameterized Gaussian distribution $D(\pi_\eta(c(t), \boldsymbol{o}_v(t)), \text{diag}(\sigma_\eta(c(t), \boldsymbol{o}_v(t))))$ where $\mu\eta$ and $\sigma\eta$ are also MLPs. In the execution phase (Fig. 4(II)), observation and reward $\boldsymbol{o}_v(t) = (X(t), r(t-1))$ are received as VRM inputs to compute internal states $c_v(t)$ using inference models. Then, the agent selects an action, sampled from $\pi\eta(\boldsymbol{a}_v(t)|c_v(t), \boldsymbol{o}_v(t))$, to interact with the environment.

B. Critic-Network Coached Service Migration Strategy Reuse Module

In this section, we introduce the Critic-network Coached service migration Strategy reuse module (CCS), a highly effective algorithm for reusing service migration strategies without the need for training any extra components. CCS is implemented based on actor-critic framework. During each iteration, CCS leverages the critic-network to create a coaching service migration strategy using both the source service migration strategies and the current target service migration strategy. After that, CCS coaches service migration strategies search by applying regularization to the target service migration strategy to mimic

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

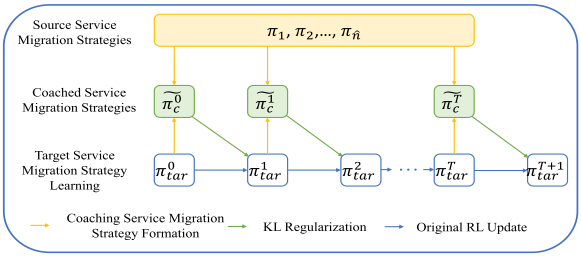

Fig. 5. The CCS algorithm. In every iteration, CCS initially generates a coaching service migration strategy by consulting the critic-network, and subsequently steers service migration strategy learning by introducing a KL regularization to the search process of service migration strategies.

the coaching service migration strategy. Next, we first present how to form a coaching service migration strategy by aggregating source service migration strategies via the critic-network. Second, we furter present the comprehensive architecture of CCS.

1) Critic-Network Coached Source Service Migration Strategy Aggregation: The CCS employs the action probabilities suggested by source service migration strategies to enhance the current target service migration strategy, resulting in the creation of a coaching service migration strategy. During iteration t of learning the target service migration strategy, for each state $\boldsymbol{o}_v(t)$ of MV agent v, the agent can access a collection of potential action probability distributions suggested by the $\hat{n}$ source service migration strategies and the current target service migration strategy: $\Pi^{\boldsymbol{o}_v(t)} = \{\pi_1(\cdot|\boldsymbol{o}_v(t)), \pi_2(\cdot|\boldsymbol{o}_v(t)), \ldots, \pi_{\hat{n}}(\cdot|\boldsymbol{o}_v(t)), \pi_{tar}^t(\cdot|\boldsymbol{o}_v(t))\}$. The coached service migration strategy π_c^t can be created by integrating the action probability distributions that exhibit the highest soft expected advantage over π_{tar}^t for each state $\boldsymbol{o}_v(t)$ of the MV agent v ($\boldsymbol{o}_v(t) \in \mathcal{O}$). This suggests that at each local observation, we can determine which source service migration strategy to adopt by merely examining its anticipated soft Q-value under π_{tar}^t.

$$\begin{aligned}\pi_c(\cdot|\boldsymbol{o}_v(t)) &= \underset{\pi(\cdot|\boldsymbol{o}_v(t))\in\Pi^{\boldsymbol{o}_v(t)}}{\arg\max} EA_{\pi_{tar}^t}(\boldsymbol{o}_v(t), \pi) \\ &= \underset{\pi(\cdot|\boldsymbol{o}_v(t))\in\Pi_t^{\boldsymbol{o}_v(t)}}{\arg\max} \mathbb{E}_{\boldsymbol{a}_v(t)\sim\pi(\cdot|\boldsymbol{o}_v(t))} \\ &\left[Q_{\pi_{tar}^t}(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t)) - \alpha \log \pi(\boldsymbol{a}_v(t)|\boldsymbol{o}_v(t))\right]. \end{aligned} \tag{17}$$

The second equation is valid because incorporating $V_{\pi_{tar}^t}(s)$ into all soft expected advantages does not alter the outcome of the $\arg\max$ operator.

2) Critic-Coached Strategy Reuse Framework: In this subsection, we present the comprehensive structure of CCS. As depicted in Fig. 5, in each iteration t, CCS initially creates a coaching service migration strategy $\widetilde{\pi}_c^t$ based on (17), Subsequently, it offers supplementary coaching to service migration strategy search by regularizing the target service migration strategy π_{tar}^{t+1} to imitate $\widetilde{\pi}_c^t$. Specifically, CCS aims to minimize the following loss function to optimize π_{tar}^{t+1},

$$\begin{aligned} L_{CCS}(\pi_{tar}^{t+1}) &= L_{actor}(\pi_{tar}^{t+1}) \\ &+ \mathbb{E}_{\boldsymbol{o}_v(t)\sim\mathcal{B}}\left[\beta_{\boldsymbol{o}_v(t)} D_{KL}\left(\pi_{tar}^{t+1}(\cdot|\boldsymbol{o}_v(t)) \,||\, \widetilde{\pi}_c^t(\cdot|\boldsymbol{o}_v(t))\right)\right], \end{aligned} \tag{18}$$

where $\beta_s > 0$ is a hyper-parameter controlling the weight of regularization and L_{actor} is the original actor loss defined in the following formula based on SAC's loss functions [34],

$$\begin{aligned} L_{critic}(Q_\lambda) &= \mathbb{E}_{(\boldsymbol{o},\boldsymbol{a},r,\boldsymbol{o}')\sim\mathcal{B}}\left[Q_\lambda(\boldsymbol{o},\boldsymbol{a}) - (r + \gamma V_{\overline{\lambda}}(\boldsymbol{o}'))\right]^2 \\ L_{actor}(\pi_\eta) &= \mathbb{E}_{\boldsymbol{o}\sim\mathcal{B}}\left[\mathbb{E}_{\boldsymbol{a}\sim\pi_\eta(\cdot|\boldsymbol{o})}\left[\alpha \log \pi_\eta(\boldsymbol{a}|\boldsymbol{o}) - Q_\lambda(\boldsymbol{o},\boldsymbol{a})\right]\right] \\ L_{entropy}(\alpha) &= \mathbb{E}_{\boldsymbol{o}\sim\mathcal{B}}\left[\mathbb{E}_{\boldsymbol{a}\sim\pi_\eta(\cdot|\boldsymbol{o})}\left[-\alpha \log \pi_\eta(\boldsymbol{a}|\boldsymbol{o}) - \alpha\overline{\mathcal{H}}\right]\right], \end{aligned}$$

where $\mathcal{B}$ is the replay buffer, $\overline{\mathcal{H}}$ is a hyper-parameter representing the target entropy, λ and η are network parameters, $\overline{\lambda}$ is target network's parameters, and $V_{\overline{\lambda}}(\boldsymbol{o}) = \mathbb{E}_{\boldsymbol{a}\sim\pi(\boldsymbol{a}|\boldsymbol{o})}[Q_{\overline{\lambda}}(\boldsymbol{o},\boldsymbol{a}) - \alpha \log \pi(\boldsymbol{a}|\boldsymbol{o})]$ is the target soft value function.

C. The Training Process of VRCCS-AC Algorithm

Algorithm 1 outlines the training procedure of the VRCCS-AC-based service migration algorithm. This algorithm includes the variational recurrent model-based actor-critic module and the critic-network coached service migration strategy reuse module. Specifically, Steps 1∼2 represent the initialization of VRCCS-AC algorithm training. Steps 3∼7 denote the gathering of training data. Steps 8∼11 and Steps 15∼16 denote the training of VRM. Steps 12∼17 denote the training of the SAC algorithm. Steps 24∼30 denote the collection of interaction experience between MV agent and target VEC scenarios. Steps 25∼38 denote the training of the CCS algorithm. Upon completion of training, the service migration strategy networks can be implemented on the MV agent to perform real-time migration decisions through neural network inference.

VI. Simulation Experiment

A. Experimental Settings

Dataset Description: We assess the service migration solution based on the VRCCS-AC algorithm using two real-world mobility traces of taxis in Rome (Italy) [35] and San Francisco (USA) [36]. In our experiments, we concentrate our examination on the core districts of Rome and San Francisco. Further, we consider deploying several edge servers (ESs) in the core area, each covering a 1 km × 1 km grid. We assign the computing capacity F_n, ($n \in \mathcal{N}$) of ES n to be 128 GHz (equivalent to four 16-core servers, with each core operating at 2 GHz). We take into account that the upload speed of real-world commercial 5G networks is generally below 60 Mbps.

Additionally, we define the upload rate $r_v^{tr}(t)$ from the closest to the farthest point within each grid to be 12∼60 Mbps, respectively. The distance between two ESs is computed using the Manhattan distance. For calculating the propagation latency, we designate the network bandwidth of the backhaul network, $b_v^{bh}(t)$, as 500 Mbps [37], and the coefficient of backhaul delay, $\sigma^{bh}(t)$, as 0.02 seconds per hop [16]. The migration delay fluctuates based on different service types and network conditions. During our training, we assume the coefficient of migration

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: The Training Process of VRCCS-AC-Based Service Migration Algorithm.

Initialize the initial-impression VRM $\mathcal{M}_f$ and the continual-learning VRM $\mathcal{M}_k$, and global counter $t \leftarrow 0$.
Initialize the network parameters, e.g., $V(\boldsymbol{o}_v(t))$ (ψ), $Q(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t))$ (λ), and the policy function $\pi(\boldsymbol{a}_v(t)|\boldsymbol{o}_v(t))$ (η). Initialize the temperature parameter α and $\mathcal{B}$.
for $T = 1$ to $|\mathcal{T}|$ **do**
  Commence an episode, set $\mathcal{M}$ with a set of initial states initialized to zero. Initialize $\boldsymbol{o}_v(t)$.
  for $t = 1$ to L **do**
    Sample a service migration strategy $\boldsymbol{a}_v(t)$ from $\pi(\boldsymbol{a}_v(t)|c_v(t),\boldsymbol{o}_v(t))$ and perform $\boldsymbol{a}_v(t)$, $t \leftarrow t + 1$.
    Collect $(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t), done(t))$ into $\mathcal{B}$.
    Calculate the next step of both VRMs using inference models.
    if $t == step_start_RL$ **then**
      For N epochs, sample a minibatch of samples from $\mathcal{B}$ to update $\mathcal{M}_f$ (15).
    end if
    if $t > step_start_RL$ and $mod(t, train_interval_KLVRM) == 0$ **then**
      Sample a minibatch of samples from $\mathcal{B}$ to update $\mathcal{M}_k$ based on typical Soft Actor-Critic by the following equations,
      $(i.)$ $J_V(\psi) = \mathbb{E}_{\boldsymbol{o}_v(t)\sim\mathcal{B}}[\frac{1}{2}(V_\psi(\boldsymbol{o}_v(t)) - \mathbb{E}_{\boldsymbol{a}_v(t)\sim\boldsymbol{\pi}_\eta}[Q_\lambda(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t)) - \alpha \log \pi_\eta(\boldsymbol{a}_v(t)|\boldsymbol{o}_v(t))])^2]$,
      $(ii.)$ $J_Q(\lambda) = \mathbb{E}_{(\boldsymbol{o}_v(t),\boldsymbol{a}_v(t))\sim\mathcal{B}}[\frac{1}{2}(Q_\lambda(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t)) - (r(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t)) + \gamma\mathbb{E}_{\boldsymbol{o}_v(t+1)\sim\mathcal{B}}[V_\psi(\boldsymbol{o}_v(t+1))]))^2]$,
      $(iii.)$ $J_\pi(\eta) = \mathbb{E}_{\boldsymbol{s}_v(t)\sim\mathcal{B}}[\mathbb{E}_{\boldsymbol{a}_\eta(\boldsymbol{o}_v(t))\sim\pi_\eta(\boldsymbol{o}_v(t))}[\alpha \log \pi_\eta(\boldsymbol{a}_\eta(\boldsymbol{o}_v(t))|\boldsymbol{o}_v(t)) - Q_\lambda(\boldsymbol{o}_v(t), \boldsymbol{a}_\eta(\boldsymbol{o}_v(t)))]]$,
      $(iv.)$ $J(\alpha) = \mathbb{E}_{\boldsymbol{o}_v(t)\sim\mathcal{B}}[\mathbb{E}_{\boldsymbol{a}\sim\pi_\eta(\boldsymbol{o}_v(t))}[-\alpha \log \pi_\eta(\boldsymbol{a}|\boldsymbol{o}_v(t)) - \alpha\mathcal{H}_{\text{tar}}]]$, $\mathcal{H}_{\text{tar}}$ denotes the target entropy.
    end if
    if $t > step_start_RL$ and $mod(t, train_interval_RL) == 0$ **then**
      Sample a minibatch of samples from $\mathcal{B}$ to update $\mathcal{R}$ (15).
    end if
  end for
end for
Require: Source service migration strategies $\{\pi_1, \pi_2, \ldots, \pi_n\}$.
while not done **do**
  for each VEC scenario step **do**
    $\boldsymbol{a}_v(t) \sim \pi_\eta$, $\boldsymbol{o}_v(t+1) \sim \boldsymbol{o}_v(t+1)|\boldsymbol{o}_v(t), \boldsymbol{a}_v(t))$.
    $\mathcal{B} \leftarrow \mathcal{B}. \cup \{\boldsymbol{o}_v(t), \boldsymbol{a}_v(t), r_v(\boldsymbol{o}_v(t), \boldsymbol{a}_v(t)), \boldsymbol{o}_v(t+1)\}$
  end for
  for each gradient step **do**
    Sample minibatch b from $\mathcal{B}$.
    Query source service migration strategies' action probabilities $\{\pi_1(\cdot|\boldsymbol{a}_v), \pi_2(\cdot|\boldsymbol{a}_v), \ldots, \pi_n(\cdot|\boldsymbol{a}_v)\}$ for states in b
    Compute expected advantages according to the soft expected advantage of action probability distribution [34], form $\widetilde{\pi_g^t}$ according to 16
    $\lambda_i \leftarrow \lambda_i - k_Q\hat{\nabla}_{\lambda_i}L_{critic}(Q_{\lambda_i})$ for $i \in \{1, 2\}$
    $\eta \leftarrow \eta - k_\pi\hat{\nabla}_\eta L_{CCS}(\pi_\eta)$
    $\alpha \leftarrow \alpha - k_\alpha\hat{\nabla}_\alpha L_{entropy}(\alpha)$
    $\overline{\lambda}_i \leftarrow \tau\lambda_i + (1-\tau)\overline{\lambda}_i$ for $i \in \{1, 2\}$
  end for
end while

TABLE III
SIMULATION EXPERIMENT PARAMETER SETTING

Parameter	Range of Values
N	16∼28
F_n	64∼150 GHz
$r_v^{tr}(t)$	12∼65 Mbps
$b_v^{bh}(t)$	500 Mbps [37]
$\sigma^{bh}(t)$	0.02 s/hop [16]
$\sigma_n(t)$	$U[1.0, 4.0]$ s/hop
$d_v^o(t)$	$U[0.05, 6]$ MB
$\lambda_v(t)$	$U[200, 11000]$ cycles/bit
$\rho_v(t)$	3 tasks/slot
$\rho_n(t)$	$U[5, 22]$ tasks/slot
B	$U[5, 10]$ Mbps

delay follows a uniform distribution within the range of $[1.0, 3.0]$ seconds per hop.

Further, we consider that both the tasks arriving at MV λ_p^u and those arriving at ES λ_p^s obey a Poisson distribution. In this experiment, we demonstrate the result of the service migration solution based on the VRCCS-AC algorithm with varying task arrival rates of MVs. The data size of offloaded tasks in practical mobile applications typically ranges from 55 KB to 6 MB [38], [39]. We assume that the data size of the tasks generated by each MV follows a uniform distribution within the interval of $[0.05, 5]$ MB. The necessary CPU cycles for each task can be computed by multiplying the data size and processing density, κ, which is uniformly distributed in the range of $[200, 10000]$ cycles/bit, encompassing tasks with varying computation complexities from low to high. We compile the parameter configurations of our simulation environment in Table III.

System Setup: We conduct all experiments on a workstation equipped with an Intel(R) Xeon(R) Silver 4210R CPU operating at 2.40 GHz, 64 GB RAM, and 2×Quadro RTX 8000 48 G GPU. All MV agents are developed using Python 3.6.

Training Setup: Our proposed service migration algorithm contains a variational recurrent model and service migration

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DETAILED HYPER-PARAMETER SETTINGS FOR VRCCS-AC

Hyperparameter	Value
γ	0.99
step_start_RL controllers	1,000
train_interval_RL	1
lr_actor	0.0003
lr_critic	0.0003
lr_α	0.0003
$\mathcal{H}_{\text{tar}}$	$-$DOF
optimizer	Adam
τ	0.005
policy_layers	256, 256
value_layers	256, 256
train_times_FIVRM model.	5,000
train_interval_KLVRM model.	5
lr_model	0.0008
seq_len update	64
batch_size update	4

strategy reuse module. The training parameters for all network components are summarized in Table IV.

The Baselines: To evaluate the performance of the VRCCS-AC algorithm, we compare the following service migration algorithms. Specifically, i) We compared traditional service migration solutions, e.g., at each time slot, an MV consistently chooses the closest ESs for migration. (Always Service Migration, ASM), The service is deployed on an ES and never migrates throughout the time period. (Never Service Migration, NSM), and At each time slot, MV randomly selects the nearest ES for service migration. (Random Service Migration, RSM). ii) Some existing service migration solutions [40], [41], [42] apply Multi-Armed Bandit (MAB) to solve the service migration and task offloading problem in partially observable environments. Further, we abbreviate these methods as MABSM. iii) In order to verify that our proposed service migration scheme based on a variational recurrent model can better solve the partially observable problem, we compare the DDPG-based service migration scheme [43] (i.e., We abbreviate this service migration solution as DRACM). Moreover, we propose a service migration algorithm based on centralized control similar to [3], [9] (i.e., RL agents can access all system status information in VEC). Further, we abbreviate this method as CACSM. iv) We assume that each MV has access to all the information in the VEC system scenario before making a service migration decision. Moreover, similar to these approaches [43], [44], we transform the service migration problem into a shortest path problem and go through an algorithm to solve it. Further, we refer to these methods simply as optimal service migration solutions (OSM). v) To validate the rapid adaptability of the VRCCS-AC algorithm to a new VEC scenario similar to these works [45], [46], we apply meta-reinforcement learning to the service migration problem. Further, we abbreviate the meta-learning-based service migration solution as METASM.

B. The Ablation Study of the VRCCS-AC Algorithm

In Fig. 6, we present the convergence curves of the VRCCS-AC algorithm. Particularly, as depicted in Fig. 6, the VRCCS-AC algorithm experiences a swift decrease in its loss metric,

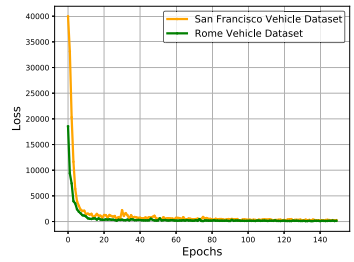

Fig. 6. Loss of the VRCCS-AC algorithm. vs. Epoch in Rome/San Francisco vehicle dataset.

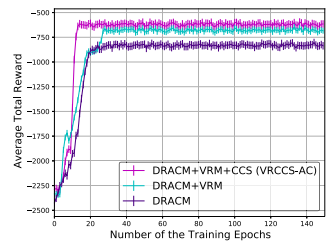

Fig. 7. Convergence and average total reward of VRCCS-AC with various optimization components in Rome vehicle dataset.

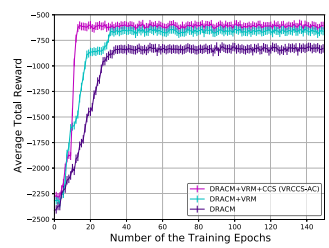

Fig. 8. Convergence and average total reward of VRCCS-AC with various optimization components in San Francisco vehicle dataset.

reaching approximately three initially. Subsequently, the loss values of the VRCCS-AC algorithm stabilize and converge to 0.5 after approximately 18 epochs. This demonstrates that greater diversity contributes to a much faster and smoother learning curve as well as our VRCCS-AC algorithm can work properly.

As demonstrated in Figs. 7 and 8, we present convergence and average total reward of VRCCS-AC with various optimization components in Rome and San Francisco vehicle datasets (For error bars, we use the standard deviation (i.e., mean $\pm$ standard deviation) form.). To confirm the effectiveness of our proposed optimization components (i.e., variational recurrent model (VRM) and Critic-Coached Strategy reuse (CCS)), we conducted the corresponding ablation experiments using the Actor-Critic-based service migration algorithm (i.e., DRACM [43]) as the baseline. Based on the DRACM, we first introduce a VRM module. As illustrated in Figs. 7 and 8, the DRACM+VRM-based service migration solution generates a higher average total reward than the DRACM. For example, in the Rome vehicle and San Francisco datasets, the average total reward of DRACM+VRM-based service migration solution is increased by approximately 25.42%. This is because a variational recurrent model (VRM) is proposed to efficiently

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EVALUATION PARAMETERS

Training Environment		Testing Environment	
N	16∼28	N	10, 15, 32
F_n	64∼150 GHz	F_n	16, 32, 180 GHz
$r_v^{tr}(t)$	12∼65 Mbps	$r_v^{tr}(t)$	10, 70, 80 Mbps
$b_v^{bh}(t)$	450∼500 Mbps [37]	$b_v^{bh}(t)$	400, 420, 550 Mbps
$\sigma^{bh}(t)$	0.02 s/hop [16]	$\sigma^{bh}(t)$	0.01, 0.03, 0.04 s/hop
$\sigma_n(t)$	$U[1.0, 4.0]$ s/hop	$\sigma_n(t)$	0.5, 6, 8 s/hop
$d_v^o(t)$	$U[0.05, 6]$ MB	$d_v^o(t)$	0.02, 8, 10 MB
$\lambda_v(t)$	$U[200, 11000]$ cycles/bit	$\lambda_v(t)$	50, 100, 13000 cycles/bit
$\rho_v(t)$	3 tasks/slot	$\rho_v(t)$	2, 6, 8 tasks/slot
$\rho_n(t)$	$U[5, 22]$ tasks/slot	$\rho_n(t)$	3, 23, 26 tasks/slot
B	$U[5, 10]$ Mbps	B	2, 15, 20 Mbps

obtain information from MV's local state through modelling VEC scenarios. An actor-critic enhances migration strategies by accessing both the VEC scenario and VRM, thus increasing the reward of the algorithm.

Second, although existing service migration schemes can achieve highly competitive migration performance when these trained service migration schemes come across a new VEC scenario, various factors such as the transmission rate, task arrival rate, computing power and coverage of ESs, and other environmental parameters may vary significantly. As a result, the migration strategies obtained by these algorithms cannot be reused. They must be optimized from scratch to adapt to new VEC scenarios, resulting in substantial computational and sampling expenses. To address this problem, based on the DRACM+VRM-based service migration solution, we introduce a CCS module. Specifically, similar to these works [45], [46], first, by varying the values of the training environment parameters in Table V, we designed three different VEC scenarios and trained three source service migration strategies using the DRACM+VRM algorithm. Second, based on these three source service migration strategies, we further train the DRACM+VRM+CCS algorithm on the new VEC scenario, i.e., the testing environment in Table V. As illustrated in Figs. 7 and 8, the DRACM+VRM+CCS-based service migration solution demonstrates faster convergence and generates higher average total reward compared to the DRACM+VRM solution, e.g., the DRACM+VRM+CCS-based service migration solution converges in the 12 epochs and in Rome vehicle and San Francisco datasets, the convergence speed of the DRACM+VRM+CCS-based service migration solution by 57.14%∼60.11%. Moreover, based on the experimental data, we calculate the p-values of the proposed model and DRACM+VRM on the Rome and San Francisco vehicle trajectories as 0.046 and 0.045, respectively. We consider the difference between the two groups to be statistically significant. This is because CCS utilizes the critic-network, a common component in actor-critic algorithm, to evaluate and choose source service migration policies. At each state, CCS chooses the source service migration strategy that has the most considerable one-step improvement over the current target service migration strategy and forms a guidance strategy. Then, the target service migration strategy is regularized to imitate the guidance service migration strategy to perform an efficient service migration strategy search. As a result, the CCS

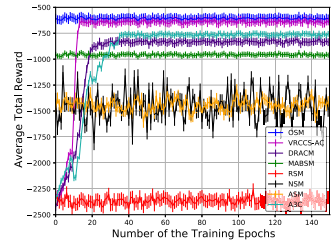

Fig. 9. Average total reward of the VRCCS-AC and other schemes in Rome vehicle dataset.

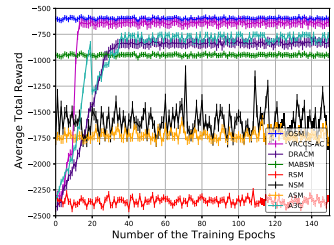

Fig. 10. Average total reward of the VRCCS-AC and other schemes in San Francisco vehicle dataset.

increases the total reward for the proposed service migration solution.

In Figs. 9 and 10, we further compare ASM, NSM, RSM, MABSM and two variants on actor-critic framework (i.e., DRACM [43], A3C [20]) service migration methods. From Figs. 9 and 10, we can observe that VRCCS-AC can converge faster and obtain higher average total rewards than the baselines; for example, in 18 epochs, the VRCCS-AC algorithm converges, and the average total rewards rise by 16.13%∼72.71%. This is due to the proposed service migration scheme can better model partially observable environments at the VEC better and extract valid information about the MV's observations, as well as the ability to reuse migration strategies obtained from previous environment training, which speeds up the convergence of the proposed migration model and improves the average total reward. Moreover, we also compare the optimal service migration scenarios in the VEC scenario. Specifically, we consider that the vehicle's travel routes and exhaustive system-level information are within knowledge during the entire episode. Further, we converted the service migration issue into an equivalent shortest path problem and addressed it utilizing Dijkstra's algorithm [43], [44]. From Figs. 9 and 10, we can observe that our proposed service migration scheme achieves near-optimal results on both vehicle trajectory datasets, with a difference of no more than 4.94% from the optimal result.

To verify that the VRCCS-AC algorithm can quickly adapt to new VEC scenarios, we propose a meta-reinforcement learning-based service migration scheme (i.e., METASM) for comparing MR-DRO algorithm [45] and DMRO algorithm [46]. As demonstrated in Figs. 11 and 12, we compared the performance of the proposed service migration scheme and METASM on both vehicle trajectory datasets. From Fig. 11, we can observe that the proposed algorithm is able to converge faster and obtain higher average total rewards than the METASM; for example,

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
THE QUANTITATIVE BASELINE COMPARISON TABLE USING MEAN OVERALL DELAY AS A METRIC ACROSS DATASETS

Datasets \ Methods	OSM	VRCCS-AC	DRACM	MABSM	METASM	RSM	NSM	ASM	CACSM
Rome	600.55±30.19	**621.92±38.38**	752.17±42.56	904.22±45.75	883.51±40.94	2250.43±90.12	1509.17±65.21	1407.88±58.36	725.44±35.52
San Francisco	588.91±25.92	**615.96±32.84**	744.39±45.86	871.697±39.68	875.51±38.51	2204.13±94.43	1453.25±67.33	1380.83±55.24	702.94±37.16

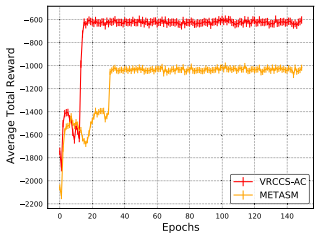

Fig. 11. Average total reward of the VRCCS-AC and METASM algorithm in Roma vehicle dataset.

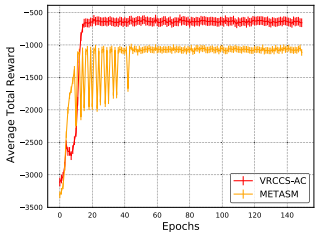

Fig. 12. Average total reward of the VRCCS-AC and METASM algorithm in San Francisco vehicle dataset.

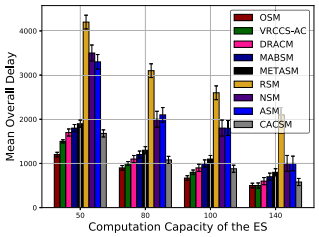

Fig. 13. Mean overall delay (second) of migration algorithms with different computation capacity of ES in Rome vehicle dataset.

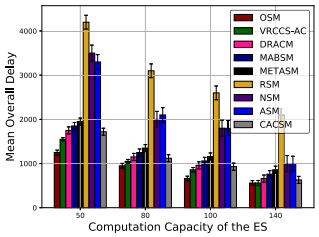

Fig. 14. Mean overall delay (s) of migration algorithms with different computation capacity of ES in San Francisco vehicle dataset.

the proposed service migration scenario has increased the convergence speed by 37.23% and the average total rewards rise by 36.51%. This is due to the fact that we introduce a service migration strategy reuse module that efficiently reuses the experience learnt by the model in different VEC scenarios and avoids the need to train any additional components, thus accelerating the training of the model and increasing the average total reward. Moreover, we also investigated the average total reward of the VRCCS-AC and METASM algorithm in the San Francisco vehicle dataset. From Fig. 12, we can find that the proposed algorithm is able to converge faster and obtain higher average total rewards than the METASM; for example, the proposed service migration scenario has increased the convergence speed by 51.22%, and the average total rewards rise by 34.13%.

C. Comparing Other Methods in Various Environmental Parameters

In Table VI, we present the quantitative baseline comparison table using mean overall delay as a metric across datasets (i.e., Rome and San Francisco vehicle datasets). As demonstrated in Table VI, we find that VRCCS-AC achieves the best service migration performance on both datasets compared to other service migration algorithms (except for the OSM algorithm). Specifically, in the Rome vehicle dataset, compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 17.29%, 31.21%, 29.61%, 72.35%, 58.73%, 55.8%, and 14.3%, respectively. In the San Francisco vehicle dataset, compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 17.26%, 29.35%, 29.65%, 72.09%, 57.62%, 55.47%, and 12.38%, respectively. This is because the VRCCS-AC can better simulate the partially observable VEC environment and extract valid information from MV observations. It is also able to reuse migration policies obtained from previous training on the environment, which results in a reduction of the mean overall delay. Moreover, compared to the optimal solution (i.e., the OSM algorithm), the VRCCS-AC algorithm obtains a mere difference of 3.43%∼4.4% migration performance for the two-vehicle trajectory datasets.

As demonstrated in Figs. 13 and 14, we display the experimental results concerning the mean overall delay of VRCCS-AC and the baseline solutions with different computation capacity of the ES using mobility traces from Rome and San Francisco, respectively. As shown in Figs. 13 and 14, the mean overall delay for each service migration algorithm decreases as the computation capacity of the MV increases. This is because as the computational resources of the ES increase, the offloaded tasks get more computational resources for a certain task arrival rate, thus reducing the total processing latency of the tasks. As demonstrated in Figs. 13 and 14, it's worth noting that we observe that VRCCS-AC attains superior service migration performance on both mobile paths in contrast to other service migration strategies. Specifically, Fig. 13 shows that compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

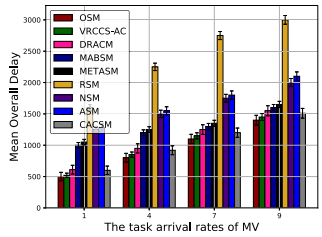

Fig. 15. Mean overall delay (s) of migration solution with different task arriving rates (tasks/slot) of MV in Rome vehicle dataset.

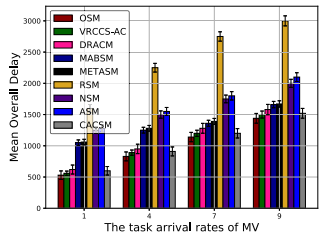

Fig. 16. Mean overall delay (second) of migration solution with different task arriving rates (tasks/slot) of MV in San Francisco vehicle dataset.

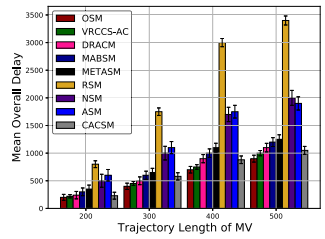

Fig. 17. Mean overall delay (second) of migration algorithms with different trajectory length of MV in Rome vehicle dataset.

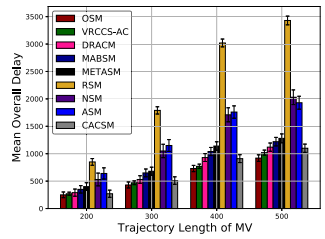

Fig. 18. Mean overall delay (s) of migration algorithms with different trajectory length of MV in San Francisco vehicle dataset.

mean overall delay of VRCCS-AC is reduced by 11.11%, 20.11%, 27.27%, 69.23%, 55.56%, 57.33%, and 10.22% respectively. Fig. 14 shows that compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 10.42%, 18.67%, 25.86%, 69.23%, 52.22%, 54.22%, and 5.42% respectively. This is due to the fact that the proposed service migration scheme is able better to model the partially observable environments of the VECs and extract valid information about the MV observations, which in turn better handles the task processing load of each ES and, therefore, drastically reduces the average total latency of the VRCCS-AC algorithm. Moreover, the VRCCS-AC attains experimental results close to the optimal values, falling within a margin of 6.25% from the optimum for both sets of mobility traces. This is due to the fact that the proposed service migration scheme is able to model the partially observable scenarios of VECs better and efficiently extract the valid information of MV observations, which in turn better balances the task processing load of each ES and, therefore, significantly reduces the average total latency of the VRCCS-AC algorithm.

As demonstrated in Figs. 15 and 16, we display the experimental results concerning the mean overall delay of VRCCS-AC and the baseline solutions under various task arrival rates of MV using mobility data from Rome and San Francisco, respectively. As shown in Figs. 15 and 16, the mean overall delay of each service migration algorithm increases as the task arrival rate of the moving vehicle increases. This is due to the fact that as the task arrival rate of MVs increases, each ES receives more offloaded tasks and will face a higher task processing load, thus increasing the latency of the service migration algorithms. As demonstrated in Figs. 15 and 16, it's worth noting that we observe that VRCCS-AC attains superior service migration performance on both mobile paths in contrast to other service migration strategies. Specifically, Fig. 15 shows that compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 7.78%, 10.77%, 14.81%, 58.18%, 20.11%, 36.11%, and 6.55% respectively. Fig. 16 shows that compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 6.25%, 11.76%, 13.67%, 56.36%, 31.43%, 33.33%, and 2.11% respectively. Moreover, the VRCCS-AC attains experimental results close to the optimal values, falling within a margin of 4.35% from the optimum for both sets of mobility traces. This is due to the fact that the proposed service migration scheme is able better to model the partially observable environments of the VECs and extract valid information about the MV observations, which in turn better handles the task processing load of each ES and, therefore, drastically reduces the average total latency of the VRCCS-AC algorithm.

As demonstrated in Figs. 17 and 18, we display the experimental results concerning the mean overall delay of VRCCS-AC and the baseline solutions with different trajectory lengths of MV using mobility traces from Rome and San Francisco, respectively. As shown in Figs. 17 and 18, the mean overall delay for each service migration algorithm decreases as the computation capacity of the MV increases. This is because as the computational resources of the ES increase, the offloaded tasks get more computational resources for a certain task arrival rate, thus reducing the total processing latency of the tasks. As demonstrated in Figs. 17 and 18, it's worth noting that we observe that VRCCS-AC attains superior service migration performance on both mobile paths in contrast to other service migration strategies. Specifically, Fig. 17 shows that compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 16.67%, 25.11%, 31.82%, 72.23%, 55.88%, 60.53%, 15.22% respectively. Fig. 18 shows that compared to DRACM, MABSM, METASM, RSM, NSM, ASM, and CACSM, the mean overall delay of VRCCS-AC is reduced by 17.20%, 25.96%, 32.46%,

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

74.83%, 54.09%, 56.82%, and 16.11% respectively. This is due to the fact that the VRCCS-AC algorithm introduces the VRM, which can be more effective at encoding underlying states from the local state data of the MV agent and, therefore, drastically reduces the average total latency of the VRCCS-AC algorithm. Moreover, the VRCCS-AC attains experimental results close to the optimal values, falling within a margin of 6.67% from the optimum for both sets of mobility traces.

VII. Conclusion

In this paper, we present a decentralized service migration approach based on the VRCCS-AC algorithm. First, a VRM is presented to attain improved data efficiency through scenario modelling. Additionally, an actor-critic enhances migration strategies by accessing both the VEC scenario and VRM. Second, CCS leverages the critic-network to assess and select source strategies. In every state, CCS selects the source strategy that exhibits the most significant one-step enhancement compared to the current target strategy and develops a coached strategy. Then, the target strategy is regularized to imitate the coached strategy to facilitate effective strategy search and efficient strategy transfer. Finally, simulation results show that VRCCS-AC can dramatically reduce latency compared to existing solutions. Moreover, VRCCS-AC can quickly adapt to new scenarios.

References

[1] C. Liu, F. Tang, Y. Hu, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, Jul. 2021.

[2] X. Chen et al., "Dynamic service migration and request routing for microservice in multicell mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13126–13143, Aug. 2022.

[3] Y. Chen, Y. Sun, C. Wang, and T. Taleb, "Dynamic task allocation and service migration in edge-cloud iot system based on deep reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 16742–16757, Sep. 2022.

[4] S. D. A. Shah, M. A. Gregory, S. Li, R. D. R. Fontes, and L. Hou, "SDN-based service mobility management in mec-enabled 5G and beyond vehicular networks," *IEEE Internet Things J.*, vol. 9, no. 15, pp. 13425–13442, Aug. 2022.

[5] X. Zhou, S. Ge, T. Qiu, K. Li, and M. Atiquzzaman, "Energy-efficient service migration for multi-user heterogeneous dense cellular networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 2, pp. 890–905, Feb. 2023.

[6] C. Wang et al., "AI-enabled spatial-temporal mobility awareness service migration for connected vehicles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3274–3290, Apr. 2024.

[7] Y. Ren, X. Chen, S. Guo, S. Guo, and A. Xiong, "Blockchain-based VEC network trust management: A DRL algorithm for vehicular service offloading and migration," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8148–8160, Aug. 2021.

[8] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.

[9] W. Chen et al., "MSM: Mobility-aware service migration for seamless provision: A data-driven approach," *IEEE Internet Things J.*, vol. 10, no. 17, pp. 15690–15704, Sep. 2023.

[10] M. Tang and V. W. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, Jun. 2022.

[11] Z. Gao, L. Yang, and Y. Dai, "Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3425–3443, Jun. 2023.

[12] A. Abouaomar, Z. Mlika, A. Filali, S. Cherkaoui, and A. Kobbane, "A deep reinforcement learning approach for service migration in mec-enabled vehicular networks," in *Proc. IEEE 46th Conf. local Comput. Netw.*, 2021, pp. 273–280.

[13] Y. Peng, L. Liu, Y. Zhou, J. Shi, and J. Li, "Deep reinforcement learning-based dynamic service migration in vehicular networks," in *Proc. IEEE Glob. Commun. Conf.*, 2020, pp. 1–6.

[14] Y. Sun, X. Guo, S. Zhou, Z. Jiang, X. Liu, and Z. Niu, "Learning-based task offloading for vehicular cloud computing systems," in *Proc. IEEE Int. Conf. Commun.*, 2018, pp. 1–7.

[15] T. Ouyang, R. Li, X. Chen, Z. Zhou, and X. Tang, "Adaptive user-managed service placement for mobile edge computing: An online learning approach," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1468–1476.

[16] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 9041–9052, Aug. 2020.

[17] B. Cao, H. Ye, J. Liu, B. Tang, Z. Tao, and S. Deng, "SMART: Cost-aware service migration path selection based on deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, to be published, doi: 10.1109/TITS.2024.3378920.

[18] H. Yu and Q. Zhang, "Hybrid learning based service migration for cost minimization with deadlines in multi-user mobile edge computing systems," *Comput. Netw.*, vol. 242, 2024, Art. no. 110249.

[19] J. Chung, K. Kastner, L. Dinh, K. Goel, A. C. Courville, and Y. Bengio, "A recurrent latent variable model for sequential data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 2980–2988.

[20] S. Tuli, S. Ilager, K. Ramamohanarao, and R. Buyya, "Dynamic scheduling for stochastic edge-cloud computing environments using A3C learning and residual recurrent neural networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 3, pp. 940–954, Mar. 2022.

[21] Q. Li, X. Ma, A. Zhou, X. Luo, F. Yang, and S. Wang, "User-oriented edge node grouping in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3691–3705, Jun. 2023.

[22] W. Zhang, J. Luo, L. Chen, and J. Liu, "A trajectory prediction-based and dependency-aware container migration for mobile edge computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3168–3181, Sep./Oct. 2023.

[23] T. Hirayama, T. Miyazawa, M. Jibiki, and V. P. Kafle, "Service function migration scheduling based on encoder-decoder recurrent neural network," in *Proc. IEEE Conf. Netw. Softwarization*, 2019, pp. 193–197.

[24] J. Baek and G. Kaddoum, "Online partial offloading and task scheduling in SDN-fog networks with deep recurrent reinforcement learning," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 11578–11589, Jul. 2022.

[25] Y. Peng et al., "Computing and communication cost-aware service migration enabled by transfer reinforcement learning for dynamic vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 257–269, Jan. 2024.

[26] Z. Zhao et al., "Reinforced-LSTM trajectory prediction-driven dynamic service migration: A case study," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 4, pp. 2786–2802, Jul./Aug. 2022.

[27] A. R. Hummaida, N. W. Paton, and R. Sakellariou, "A hierarchical decentralized architecture to enable adaptive scalable virtual machine migration," *Concurrency Comput.: Pract. Experience*, vol. 35, no. 2, 2023, Art. no. e7487.

[28] K. Shuai, Y. Miao, K. Hwang, and Z. Li, "Transfer reinforcement learning for adaptive task offloading over distributed edge clouds," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 2175–2187, Second Quarter 2023.

[29] M. Teng, X. Li, and K. Zhu, "Joint optimization of sequential task offloading and service deployment in end-edge-cloud system for energy efficiency," *IEEE Trans. Sustain. Comput.*, to be published, doi: 10.1109/TSUSC.2023.3291365.

[30] Y. Ma et al., "A performance and reliability-guaranteed predictive approach to service migration path selection in mobile computing," *IEEE Internet Things J.*, vol. 10, no. 20, pp. 17977–17987, Oct. 2023.

[31] N. Lin, H. Tang, L. Zhao, S. Wan, A. Hawbani, and M. Guizani, "A PDDQNLP algorithm for energy efficient computation offloading in UAV-assisted MEC," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 8876–8890, Dec. 2023.

[32] H. Hu, D. Wu, F. Zhou, X. Zhu, R. Q. Hu, and H. Zhu, "Intelligent resource allocation for edge-cloud collaborative networks: A hybrid DDPG-D3QN approach," *IEEE Trans. Veh. Technol.*, vol. 72, no. 8, pp. 10696–10709, Aug. 2023.

[33] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.

[34] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 1861–1870.
[35] L. Bracciale, M. Bonola, P. Loreti, G. Bianchi, R. Amici, and A. Rabuffi, "CRAWDAD dataset roma/taxi (v.2014–07-17)," Jul. 2014. [Online]. Available: https://crawdad.org/roma/taxi/20140717
[36] M. Piorkowski, N. Sarafijanovic-Djukic, and M. Grossglauser, "Crawdad Data Set epfl/mobility (V. 2009–02-24)," 2009.
[37] L. Ma, S. Yi, N. Carter, and Q. Li, "Efficient live migration of edge services leveraging container layered storage," *IEEE Trans. Mobile Comput.*, vol. 18, no. 9, pp. 2020–2033, Sep. 2019.
[38] D. C. Nguyen, P. N. Pathirana, M. Ding, and A. Seneviratne, "Privacy-preserved task offloading in mobile blockchain with deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2536–2549, Dec. 2020.
[39] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
[40] Y. Sun, S. Zhou, and J. Xu, "EMM: Energy-aware mobility management for mobile edge computing in ultra dense networks," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 11, pp. 2637–2646, Nov. 2017.
[41] T. Ouyang, R. Li, X. Chen, Z. Zhou, and X. Tang, "Adaptive user-managed service placement for mobile edge computing: An online learning approach," in *Proc. IEEE Conf. Comput. Commun.*, 2019, pp. 1468–1476.
[42] X. Wang, J. Ye, and J. C. Lui, "Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1199–1208.
[43] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2333–2345, Oct. 2018.
[44] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. S. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Trans. Mobile Comput.*, vol. 20, no. 3, pp. 939–951, Mar. 2021.
[45] Z. Zhang, N. Wang, H. Wu, C. Tang, and R. Li, "MR-DRO: A fast and efficient task offloading algorithm in heterogeneous edge/cloud computing environments," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3165–3178, Feb. 2023.
[46] G. Qu, H. Wu, R. Li, and P. Jiao, "DMRO: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 3, pp. 3448–3459, Sep. 2021.

Zhen Gao received the MS degree from the School of Computer Science and Engineering, Northeastern University, Shenyang, China, in 2020. His research interests include MEC, task offloading, and reinforement learning.

Lei Yang received the PhD degree in computer application and technology from Northeastern University, China, in 2008. He is an associate professor with the School of Computer Science and Engineering, Northeastern University, China. His current research interests include Big Data and cloud computing.

Yu Dai received the PhD degree in computer application and technology from Northeastern University, China, in 2008. She is an associate professor with the College of Software, Northeastern University, China. Her current research interests include Big Data, cloud computing, and intelligent web information system.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 02,2026 at 09:55:01 UTC from IEEE Xplore. Restrictions apply.